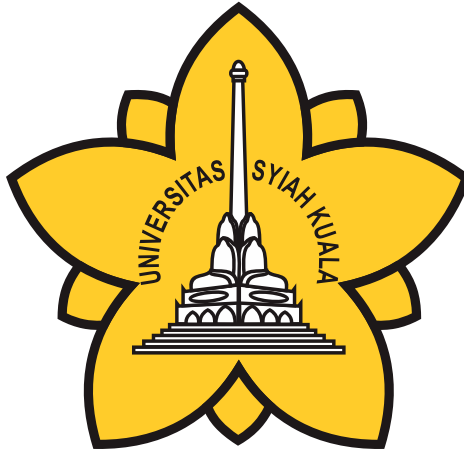


PROJECT BIG CHALLENGE

Dosen Pengampu :

Prof. Dr. Taufik Fuadi Abidin, S.Si., M.Tech

197010081994031002



Disusun oleh :

Virzi Mayhand Syahputra

: 250810701100077

Rafid Khaulika

: 250810701100016

MATA KULIAH PEMROGRAMAN

DEPARTEMEN INFORMATIKA PRODI INFORMATIKA

FAKULTAS MATEMATIKA DAN ILMU PENGETAHUAN ALAM

UNIVERSITAS SYIAH KUALA

DARUSSALAM - BANDA ACEH

KATA PENGANTAR

Puji dan syukur penulis panjatkan ke hadirat Allah SWT atas rahmat dan karunia-Nya sehingga laporan *Project Big Challenge* pada mata kuliah Pemrograman ini dapat diselesaikan dengan baik dan tepat waktu. Laporan ini disusun sebagai bentuk pertanggungjawaban akademik sekaligus sarana penerapan konsep dan logika pemrograman menggunakan bahasa C.

Laporan ini membahas perancangan dan implementasi program yang mampu membaca serta memproses data dari file teks, melakukan parsing dan pembersihan data, menghitung serta mengelompokkan kata berdasarkan abjad dan frekuensi, hingga menyimpan hasil pengolahan ke dalam file binari yang dapat dibaca kembali. Melalui project ini, penulis diharapkan dapat memahami penerapan struktur data, algoritma pengurutan, serta pengelolaan file teks dan file binari.

Penulis mengucapkan terima kasih kepada Prof. Dr. Taufik Fuadi Abidin, S.Si., M.Tech selaku dosen pengampu mata kuliah Pemrograman atas arahan dan bimbingan yang telah diberikan. Ucapan terima kasih juga disampaikan kepada rekan-rekan yang telah memberikan dukungan selama pengerjaan project ini.

Penulis menyadari bahwa laporan ini masih memiliki keterbatasan, sehingga kritik dan saran yang membangun sangat diharapkan. Akhir kata, semoga laporan ini dapat memberikan manfaat bagi pembaca dalam memahami penerapan pemrograman C dan pengolahan data secara efisien.

Banda Aceh, 28 Desember 2025

Penyusun

DAFTAR ISI

KATA PENGANTAR	i
DAFTAR ISI	ii
BAB I PERMASALAHAN	1
BAB II METODE SCORING.....	2
2.1. Desain Program	2
2.2. Flowchart Program	3
2.3. Struktur Data	4
2.4. Proses Parsing dan Pembersihan Data.....	5
2.5. Algoritma Pengurutan	6
2.6. Penyimpanan dan Pembacaan File Binari	7
BAB III HASIL	8
3.1. Kondisi 1	8
3.2. Kondisi 2	9
3.3. Kondisi 3	9
REFERENSI	11
LAMPIRAN	12

BAB I PERMASALAHAN

Project Big Challenge ini bertujuan untuk membangun sebuah program berbahasis bahasa pemrograman ANSI C yang mampu membaca dan memproses data dari sebuah file teks. Dengan format program yang berbentuk modular, program ini diminta untuk mengabaikan seluruh strings yang berada dalam tag `<url> ... </url>` serta menghimpun setiap kata yang berada di dalam tag `<title> ... </title>` dan `<body> ... </body>` yang telah dibersihkan dari tanda baca, dibersihkan dari digit, dan diubah menjadi huruf kecil (*lower-case*).

Data kata yang telah dihimpun kemudian diurutkan berdasarkan frekuensi kemunculan dari terbesar ke terkecil (*descending order*) per hurufnya. Jika terdapat kata dengan frekuensi kemunculan yang sama pada huruf yang sama, maka pengurutan dilakukan berdasarkan panjang kata dari terbesar ke terkecil, dan jika masih sama, maka akan diurutkan berdasarkan urutan alphabet secara menurun.

Hasil pengurutan itu kemudian disimpan dalam sebuah file binari dengan format yang telah ditentukan, dapat dilakukan secara *hardcoded* kedalam kode program, diinputkan dan ditanyakan kepada user saat program dijalankan, atau boleh juga melalui command line argument. Dengan susunan sebagai berikut:

[abjad][jumlah_kata_dengan_abjad_tersebut][panjang_kata][kata][frekuensi]
[panjang_kata][kata][frekuensi][panjang_kata][kata][frekuensi]... dstnya sebanyak
jumlah kata dengan abjad tersebut dari a-z

Dimana:

[abjad]	bertipe char, meliputi {a,b,c, ...,z}
[jumlah_kata]	bertipe integer
[panjang_kata]	bertipe integer
[kata]	bertipe array of char

Program diminta menyediakan menu untuk menyimpan setiap kata yang sudah di urutkan ke dalam file binary serta memiliki pilihan untuk menampilkan maksimum n buah kata ($0 < n \leq 25$) dengan frekuensi terbesar yang dibaca kembali dari file binari

BAB II METODE SCORING

Metode scoring merupakan metode penyelesaian dan logika pengolahan data yang digunakan dalam program Big Challenge untuk menentukan bagaimana kata-kata hasil pemrosesan dinilai, dikelompokkan, dan diurutkan sebelum disimpan ke dalam file binari dan ditampilkan ke layar monitor.

2.1. Desain Program

Program ini dirancang secara modular dengan memisahkan setiap fungsi ke dalam beberapa *source code* yang berbeda untuk memudahkan pengembangan dan *maintenance* kode. Struktur modular ini terdiri dari file utama dan beberapa file pendukung yang menangani fungsi tertentu seperti parsing data, pembacaan file binari, pengelolaan struktur data hingga tampilan output yang dihasilkan oleh programnya.

Pada program ini terdapat 4 directory yang berbeda fungsinya, antara lain seperti directory *src* yang menyimpan seluruh *source code* dari program ini, *include* yang menampung file-file header untuk menyambungkan antar file *source code*, directory *data* yang menyimpan file teks untuk dijadikan input ke dalam programnya, serta *bin* yang berfungsi sebagai tempat disimpannya file binari yang sudah dibuat oleh program. Berikut adalah program yang digunakan:

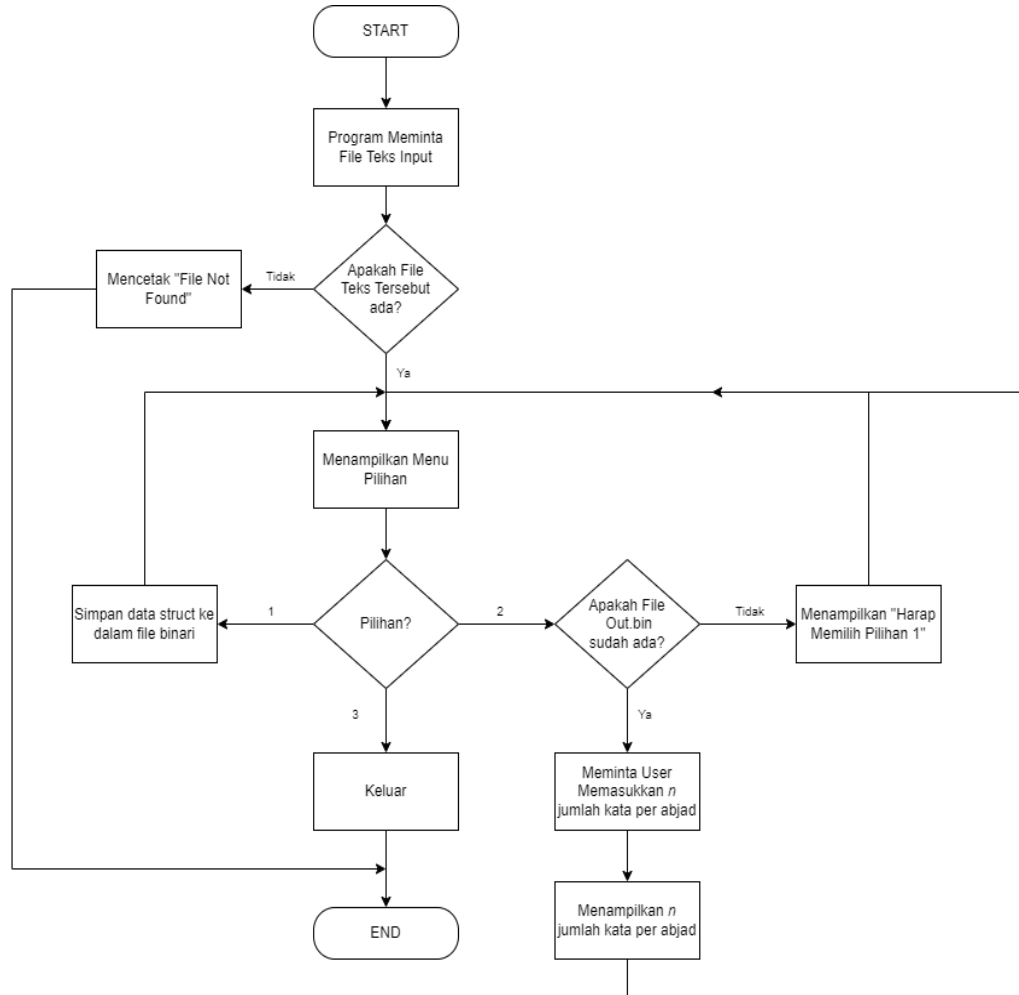


```
PS D:\belajarC\Kuliah\bersamaProfTaufik\Tugas\Big-Challenge-Prof> tree /F
Folder PATH listing for volume InDe
Volume serial number is 48B8-6781
D:..
|   makefile
|   README.md
|
|--- bin
|
|--- data
|    1k.txt
|    50k.txt
|
|--- include
|    fileHandling.h
|    globals.h
|    parsing.h
|    structure.h
|    view.h
|
|--- src
|    fileHandling.c
|    main.c
|    parsing.c
|    structure.c
|    view.c
```

Gambar 1 Desain Program Big-Challenge

2.2. Flowchart Program

Untuk menunjukkan alur kerja program Project Big-Challenge secara keseluruhan, dimulai dari proses input file teks, pengecekan keberadaan file, pemilihan menu hingga penyimpanan data kedalam file binari dan menampilkan hasil ke monitor.



Gambar 2 Alur Kerja Program Big-Challenge

2.3. Struktur Data

Program ini menggunakan **struct** untuk merepresentasikan setiap kata yang diproses. Struktur ini menyimpan beberapa informasi penting, yaitu huruf awal kata sebagai abjad, panjang kata, kata itu sendiri, dan frekuensi kemunculan kata tersebut. **Struct** yang digunakan dalam program ini juga sudah diberikan alias menggunakan **typedef** dengan nama `StoringWordsInfo`. Berikut merupakan **struct** yang digunakan dalam program ini:

```
typedef struct {  
    char abjad;  
    int panjangKata;  
    char kata[MAX_WORD_LEN];  
    int frekuensi;  
} StoringWordsInfo;
```

Gambar 3 Struct Utama Big-Challenge

Dalam pengaplikasian `StoringWordsInfo` dalam project ini, kami mendeklarasikan sebuah array **listKataPerHuruf** yang merupakan array 2D. Indeks baris diisi oleh abjad kata yang sedang di proses dan indeks kolom diisi oleh maximal kata yang boleh disimpan dalam satu abjad. Di dalam program ini juga di deklarasikan sebuah array **jumlahKataUnikPerAbjad** yang berfungsi untuk menyimpan seberapa banyak kata unik yang ada di dalam suatu abjad. Ditunjukkan pada gambar dibawah ini:

```
StoringWordsInfo listKataPerHuruf[ALPHABET_SIZE][MAX_WORD_PER_ALPHABETS];  
int jumlahKataUnikPerAbjad[ALPHABET_SIZE] = {0};
```

Gambar 4 Deklarasi Array

Dapat dilihat pada Gambar 4 bahwa `listKataPerHuruf` dan `jumlahKataPerAbjad` menggunakan **ALPHABET_SIZE** yang merupakan sebuah konstanta 26 yang merepresentasikan setiap alphabet dalam bentuk *integer*. Sedangkan **MAX_WORD_PER_ALPHABET** merupakan konstanta yang bernilai 16000 yang digunakan untuk membatasi setiap kata yang dapat disimpan dalam satu abjad tertentu. Nilai 16000 dipilih karena setelah melihat

hasil dari jumlahKataUnikPerAbjad, tidak ada yang melebihi nilai 16000 seperti yang terlihat dibawah ini:

```
$ ./program.exe
Masukkan nama file input: data\50k.txt
6935  9639  5028  9490  2394  3095  3741  3478  2994
2643  9033  4003  15819 3202  1591  11550 279  4564
11819 7403  1313  1486  1979  201  1049  665
```

Gambar 5 Jumlah Kata per Abjad

Nilai tertinggi merupakan 15819, sehingga pemilihan nilai maksimum 16000 dapat menyimpan setiap kata yang terhimpun dalam abjad tertentu dan juga menghindari jika terjadi *overflow data* yang menyebabkan programnya *crash*.

2.4. Proses Parsing dan Pembersihan Data

Proses pertama dalam segmen ini adalah mengambil kalimat yang hanya ada pada tag body dan tiitle serta mengabaikan tag url. Untuk tag *url* kami menggunakan fungsi **strstr()** yang digunakan untuk mencari kemunculan pertama tag tersebut dalam sebuah string teks kemudian melewatinya sebanyak 5 karakter untuk tag pembuka dan 6 karakter untuk tag penutup.

Sedangkan untuk tag body dan tiitle, kami menggunakan fungsi **strncmp()** dan hanya mengambil strings setelah tag pembuka dan berhenti setelah tag penutup ditemukan.

Setelah tag body, url, dan tiitle tidak lagi ditemukan, maka akan dilakukan pembersihan strings dari symbols dan digit yang kemudian diubah menjadi huruf kecil (lowercase). Pada proses ini kami hanya menggunakan fungsi **isalpha()**, karena fungsi ini otomatis hanya mengambil alphabets saja, karena kami juga tidak ingin ada escape sequence seperti `\n`, `\t`, `\r` masuk kedalam teks yang akan diolah dan menyebabkan masalah tokenisasi berikutnya.

Selanjutnya adalah tokenisasi, yaitu pemecahan menjadi kata-kata tunggal. Proses tokenisasi dilakukan menggunakan fungsi **strtok()**, dengan spasi dan *newline* sebagai *delimiter*.

2.5. Algoritma Pengurutan

Dalam Project Big-Challenge ini algoritma pengurutan yang digunakan adalah **Insertion Sort**, yaitu algoritma sederhana yang bekerja dengan membangun daftar terurut satu persatu, di mana elemen dari bagian yang belum terurut disisipkan ke posisi yang tepat di bagian yang sudah terurut.

Proses pengurutan kata dilakukan per abjad, dikarenakan jumlah kata unik setiap abjadnya sudah diketahui, sehingga untuk melakukan pengurutan hanya perlu melakukan perulangan sampai dengan seberapa banyak kata unik di abjad tersebut.

Pada proses ini pengurutan dilakukan berdasarkan kriteria yang sudah ditetapkan pada soal, yaitu secara *descending order* per hurufnya. Kata yang memiliki frekuensi lebih banyak akan berada lebih dahulu, jika terdapat kata dengan frekuensi kemunculan yang sama pada huruf yang sama, maka pengurutan dilakukan berdasarkan panjang kata dari terbesar ke terkecil, dan jika masih sama, maka akan diurutkan berdasarkan urutan alphabet secara menurun.

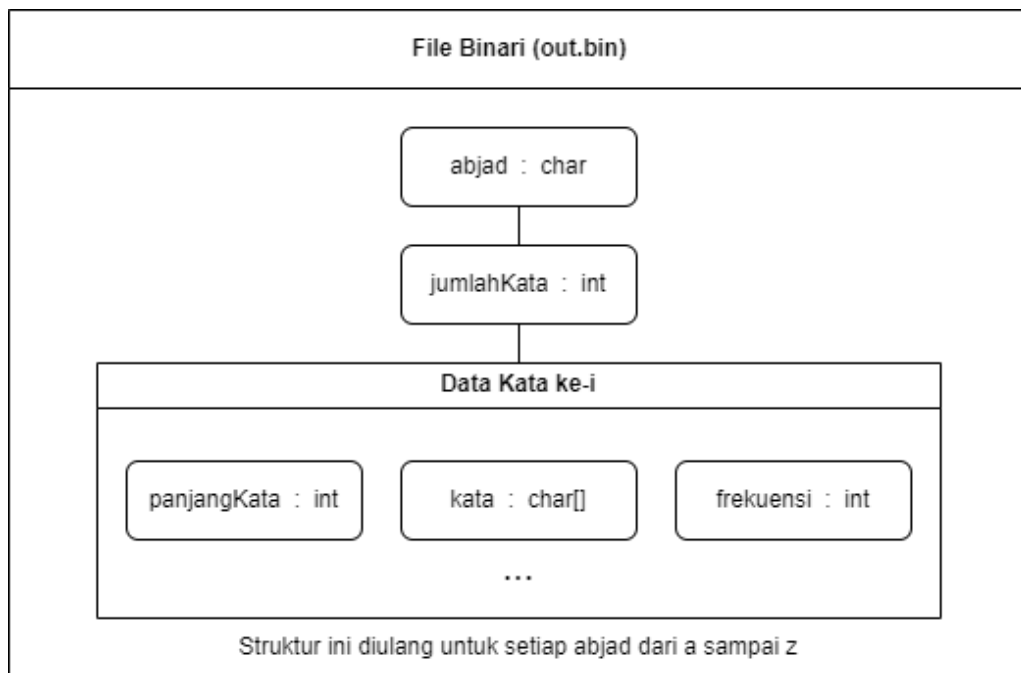
Untuk contoh metode ini dapat dilihat dari tabel dibawah ini:

Kata dengan awalan P			
Kata	Frekuensi	Panjang	Abjad
Positif	10	7	
Pisang	10	6	
Panti	8	5	$t > c$
Panci	8	5	$c < t$
Paus	5	4	

Tabel 1 Contoh Penyortiran Urutan Kata

2.6. Penyimpanan dan Pembacaan File Binari

Setelah seluruh data berhasil diproses dan diurutkan, tahap selanjutnya adalah melakukan penyimpanan data. Kami menggunakan fungsi ***fwrite()*** dengan mode ***wb*** dalam project ini untuk menulis ke dalam file binari yang dilakukan dengan menuliskan abjad, jumlah kata dalam abjad tersebut dan melakukan perulangan untuk setiap kata dalam abjad a sampai z, panjang kata serta frekuensinya. Berikut adalah struktur penyimpanan data di dalam file binari:



Gambar 6 Struktur Penyimpanan di dalam File Binari

Untuk proses membaca kembali data dari file binari, menggunakan fungsi ***fread()*** dengan mode ***rb*** adalah pilihan kami pada Project Big-Challenge ini. Program akan membaca abjad dan jumlah kata unik dalam abjad tersebut, setelah itu program akan melakukan pengulangan sebanyak jumlah kata unik dalam abjad tersebut dengan menampilkan panjang kata unik, kata unik itu sendiri, dan frekuensinya yang akan ditampilkan ke layar.

BAB III HASIL

Pada bagian ini akan dilakukan pengujian terhadap program yang telah dibuat untuk memastikan seluruh fungsionalitas berjalan sesuai arahan Project Big-Challenge. Pengujian akan dilakukan dengan beberapa kondisi pemilihan menu yang berbeda. Setiap pengujian dilakukan saat tidak ada file **out.bin** yang memiliki tujuan untuk menguji alur program, logika menu dan kestabilan program dalam menangani input.

Proses *compile* program dan menentukan file teks sebagai input:

```
PIJII@DESKTOP-BGDB3GM MINGW64
/d/belajarc/kuliah/bersamaProfTaufik/Tugas/Big-Challenge-Prof
$ make
gcc -Wall ./src/main.c ./src/fileHandling.c ./src/parsing.c
./src/structure.c ./src/view.c -o program

PIJII@DESKTOP-BGDB3GM MINGW64
/d/belajarc/kuliah/bersamaProfTaufik/Tugas/Big-Challenge-Prof
$ ./program.exe
Masukkan nama file input: data\50k.txt
[+] File sedang dalam proses penyortiran. Harap menunggu
```

3.1. Kondisi 1

User memilih pilihan 1, kemudian pilihan 2 (5 kata), lalu memilih pilihan 3:

```
Menu:
1. Simpan ke file binari
2. Tampilkan n kata per abjad
3. Keluar
Pilihan Anda: 1
[+] File sedang disimpan. Harap menunggu
[+] Data berhasil disimpan ke out.bin

Menu:
1. Simpan ke file binari
2. Tampilkan n kata per abjad
3. Keluar
Pilihan Anda: 2
Tampilkan berapa kata (0<n<=25): 5
a {akan (90115), ada (31915), asal (23166), adalah (23156), atas (15193)}
b {bisa (54442), belum (24545), bermain (23757), baru (20428), baik (20423)}
c {com (31762), chelsea (11948), cedera (11315), city (11174), cukup (10727)}
d {di (253431), dan (154697), dengan (116219), dari (76165), dalam (58891)}
e {empat (11002), emas (6517), eropa (5935), enam (4771), euro (3388)}
f {final (12179), fc (5600), fokus (4760), februari (4011), fans (3970)}
g {gol (27591), gelar (13491), gelandang (8965), gagal (8501), gawang (8012)}
h {harus (31218), hanya (26686), hal (18602), hari (16118), hasil (15323)}
i {ini (121544), itu (113142), indonesia (29604), ia (26127), ingin (18729)}
j {juga (44251), jika (21550), juara (19941), jadi (16593), jakarta (8253)}
k {ke (59578), kami (56883), klub (31677), karena (30599), kedua (26024)}
l {laga (37072), lebih (30464), liga (26945), lalu (25474), lagi (15230)}
m {musim (46744), menjadi (44875), mereka (39316), masih (33531), membuat (22734)}
n {namun (31282), nanti (14193), nama (7081), nasional (6398), nomor (5915)}
o {oleh (17212), orang (8240), olahraga (6587), olimpiade (5507), oktober (3766)}
p {pemain (93900), pada (70134), pelatih (40925), pertandingan (36789), pertama (23227)}
q {qatar (1146), qpr (911), qnb (597), queens (418), q (159)}
r {rossi (12354), rumah (9527), real (8235), resmi (7810), rabu (7260)}
s {saya (89080), saat (47705), sudah (39794), seperti (35156), setelah (35037)}
t {tidak (78438), tim (68894), tersebut (44732), tahun (36674), tak (30436)}
u {untuk (109657), united (15625), utama (9389), ujar (9133), usai (8899)}
v {van (5118), vs (4604), valentino (4037), valencia (3285), v (2368)}
w {wib (27777), waktu (15979), wenger (3867), wasit (3735), wakil (3674)}
x {xix (949), xavi (874), x (649), xviii (622), xabi (295)}
y {yang (229474), yakni (7621), yakin (6843), yamaha (5317), ya (2829)}
z {zimbo (3049), zona (1409), zlatan (445), zanetti (428), zidane (313)}

Menu:
1. Simpan ke file binari
2. Tampilkan n kata per abjad
3. Keluar
Pilihan Anda: 3
[+] Program Selesai.
```

Gambar 7 Output Program Kondisi Pertama

3.2. Kondisi 2

User memilih pilihan 1, kemudian pilihan 2 (0 kata), lalu memilih pilihan 3:

```
Menu:
1. Simpan ke file binari
2. Tampilkan n kata per abjad
3. Keluar
Pilihan Anda: 1
[+] File sedang disimpan. Harap menunggu
[+] Data berhasil disimpan ke out.bin

Menu:
1. Simpan ke file binari
2. Tampilkan n kata per abjad
3. Keluar
Pilihan Anda: 2
Tampilkan berapa kata (0<n<=25): 0
a {}
b {}
c {}
d {}
e {}
f {}
g {}
h {}
i {}
j {}
k {}
l {}
m {}
n {}
o {}
p {}
q {}
r {}
s {}
t {}
u {}
v {}
w {}
x {}
y {}
z {}

Menu:
1. Simpan ke file binari
2. Tampilkan n kata per abjad
3. Keluar
Pilihan Anda: 3
[+] Program Selesai.
```

Gambar 8 Output Program Kondisi Kedua

Kondisi 3.2 ini dilakukan atas arahan Prof Taufik pada saat mata kuliah pemrograman berlangsung di dalam kelas.

3.3. Kondisi 3

User memilih pilihan 2, lalu memilih pilihan 3:

```
Menu:
1. Simpan ke file binari
2. Tampilkan n kata per abjad
3. Keluar
Pilihan Anda: 2
[!] File out.bin belum ada, harap pilih opsi 1 terlebih dahulu

Menu:
1. Simpan ke file binari
2. Tampilkan n kata per abjad
3. Keluar
Pilihan Anda: 3
[+] Program Selesai.
```

Gambar 9 Output Program Kondisi Ketiga

Di dalam program Project Big-Challenge kami juga menggunakan fungsi khusus yaitu ***fflush()*** untuk meneruskan stream ***stdout*** langsung ke layar karena saat kami mencoba print peringatan sebuah proses, terjadi *stuck* saat program berjalan.

Link video youtube:

[iniLinkMwehehehe](#)

REFERENSI

GeeksforGeeks. (n.d.). *strstr() in C/C++*. GeeksforGeeks.

<https://www.geeksforgeeks.org/cpp/strstr-in-ccpp/>

GeeksforGeeks. (n.d.). *strtok() and strtok_r() functions in C with examples*.

GeeksforGeeks. https://www.geeksforgeeks.org/cpp/strtok-strtok_r-functions-c-examples/

W3Schools. (n.d.). *C ctype.php Reference*. W3Schools.

https://www.w3schools.com/c/c_ref_ctype.php

GeeksforGeeks. (n.d.). *Insertion sort algorithm*. GeeksforGeeks.

<https://www.geeksforgeeks.org/dsa/insertion-sort-algorithm/>

GeeksforGeeks. (2014, 21 Juli). *Insertion sort* [Video]. YouTube.

<https://www.youtube.com/watch?v=Tz7vBodZqo8>

Coffee and Codes. (2020, 26 Agustus). *Understanding insertion sort: A step-by-step guide with complexity analysis*. Medium.

<https://medium.com/coffee-and-codes/understanding-insertion-sort-a-step-by-step-guide-with-complexity-analysis-54592340a3c1>

Kerrisk, M. (n.d.). *fflush(3) — Linux manual page*. Linux man-pages project.

<https://man7.org/linux/man-pages/man3/fflush.3.html>

Stack Overflow. (2011, 4 Maret). *What are file descriptors, explained in simple terms?* [Komentar forum daring]. <https://stackoverflow.com/questions/5256599/what-are-file-descriptors-explained-in-simple-terms>

GeeksforGeeks. (n.d.). *fread() function in C*. GeeksforGeeks.

<https://www.geeksforgeeks.org/c/fread-function-in-c/>

GeeksforGeeks. (n.d.). *fwrite() in C*. GeeksforGeeks.

<https://www.geeksforgeeks.org/c/fwrite-in-c/>

LAMPIRAN

Repository Project Big-Challenge:

<https://github.com/virzimayhandsyahputra/Big-Challenge-Prof>